

Cheese Data Analysis

```
cheese_df <- read.csv("Cheese_Chemical_Data.csv")
head(cheese_df)
```

##	Sample	Class	PrimaryID	Feature1_N	Feature2_N	Feature3_N	Feature4_N	Feature5_N
## 1	B	B	B1	1163170.0	27168800	2094430	1347.45	2115440
## 2	B	B	B2	1194870.0	36163600	2152660	1553.65	2368630
## 3	B	B	B3	857268.0	26036600	1874090	1250.21	1965370
## 4	B	B	B4	1038150.0	33538100	1768120	1188.42	2030960
## 5	H	H	H1	99992.5	12860400	302512	16147.20	2921580
## 6	H	H	H2	101257.0	13372200	277844	16420.60	3150310
##	Feature6_N	Feature7_N	Feature8_N	Feature9_N	Feature10_N	Feature11_N		
## 1	50.5272	2657.22	192940.0	323.737	30938.6	2927.79		
## 2	66.0252	3020.81	221871.0	353.542	34604.2	2908.27		
## 3	82.3999	2469.24	174869.0	279.029	28636.7	2995.98		
## 4	56.3799	2462.78	162947.0	267.196	26981.6	2576.16		
## 5	714.8130	2283.43	38478.2	3203.000	22844.7	17557.00		
## 6	645.6920	2304.05	30479.6	2644.750	24791.5	14033.50		
##	Feature12_N	Feature13_N	Feature14_N	Feature15_N	Feature16_N	Feature17_N		
## 1	234.145	154.7430	67620.8	4699.770	51689.9	17.62830		
## 2	158.936	133.8520	76767.6	5629.230	53711.3	21.12480		
## 3	165.353	128.7640	78207.5	4436.480	48191.9	8.85583		
## 4	212.884	101.2790	69120.9	4077.440	39712.8	16.48860		
## 5	230.743	44.5211	47664.2	896.727	10320.4	444.28400		
## 6	125.796	172.3900	49385.9	867.701	11099.1	591.33800		
##	Feature18_N	Feature19_N	Feature20_N	Feature21_N	Feature22_N	Feature23_N		
## 1	2603.37	4461540	95483.2	14611.400	220589	152657.000		
## 2	3298.35	4751330	91125.4	16408.000	215358	171126.000		
## 3	3066.20	4448680	95469.0	14690.200	219693	171371.000		
## 4	2548.90	4142610	77161.7	12751.600	161957	147899.000		
## 5	1621.21	211075	20123.4	431.659	116452	374.436		
## 6	1883.15	194975	32523.7	468.346	147289	318.159		
##	Feature24_N	Feature25_N	Feature26_N	Feature27_N	Feature28_N	Feature29_N		
## 1	17265.8	14482.50	73332.9	679.112	3958.81	21454.7		
## 2	19666.9	7842.21	79811.5	795.313	5367.02	18161.5		
## 3	18331.8	10005.90	100705.0	1272.930	6836.26	21072.7		
## 4	15599.9	10486.50	97089.7	841.841	5789.48	19679.0		
## 5	110453.0	459203.00	13798.9	3867.250	3662.68	12005.3		
## 6	108854.0	463903.00	30928.3	5320.130	3440.08	10667.3		
##	Feature30_N	Feature31_N	Feature32_N	Feature33_N	Feature34_N	Feature35_N		
## 1	3608.93	2582.76	2657.93	23513.60	10381.10	2221.820		
## 2	3845.54	9445.49	2278.92	24288.10	6875.61	1016.060		
## 3	5355.22	13284.80	2675.48	25108.30	8047.31	864.733		
## 4	3812.53	8147.22	2045.30	20822.90	5058.71	849.688		
## 5	496886.00	3306.94	1405.94	7611.99	114826.00	1182.910		
## 6	495019.00	4096.64	2778.13	7502.68	117363.00	659.990		
##	Feature36_N	Feature37_N	Feature38_N	Feature39_N	Feature40_N	Feature41_N		
## 1	482.261	5341.550	36803.8	3639.7200	1760.4100	28657.40		
## 2	394.555	5210.760	36487.5	3504.2500	1689.3400	28998.00		
## 3	718.637	5474.220	35457.6	2868.3500	1369.3300	24685.90		
## 4	445.572	5736.550	36139.6	3057.2600	1483.9900	26794.10		
## 5	838.446	827.678	25922.0	37.0503	27.0002	2733.92		
## 6	677.150	2008.570	31216.3	34.7184	46.2406	3226.86		

```
##      Feature42_N Feature43_N Feature44_N Feature45_N Feature46_N Feature47_N
## 1      37.5396      351.966      9032500      1204.710      97.9231      5.04725
## 2      31.2879      400.467      8235580      1060.900      148.3260      26.04960
## 3      31.9741      303.432      7569210      893.061      175.5560      3.35137
## 4      24.7417      324.208      8340590      998.615      70.4434      6.14720
## 5      10.9559      180.690      251271      188.813      6402.9800      18.92670
## 6      18.7334      186.193      335151      217.333      7006.0000      36.61170
##      Feature48_N Feature49_N Feature50_N Feature51_N
## 1  1010440.00      1116690      16379.5      2126.890
## 2   886227.00      920069      13455.1      1498.280
## 3   807323.00      790638      11645.0      993.516
## 4   863928.00      853713      12503.6      1495.040
## 5    3672.75     4055540      22301.1     13260.700
## 6    4143.28     4218670      23591.9     15145.500
```

Data Validation

Checking for null values

```
cheese_df[!complete.cases(cheese_df),]
```

```
## [1] Sample      Class      PrimaryID   Feature1_N Feature2_N Feature3_N
## [7] Feature4_N   Feature5_N Feature6_N   Feature7_N Feature8_N Feature9_N
## [13] Feature10_N  Feature11_N Feature12_N  Feature13_N Feature14_N Feature15_N
## [19] Feature16_N  Feature17_N Feature18_N  Feature19_N Feature20_N Feature21_N
## [25] Feature22_N  Feature23_N Feature24_N  Feature25_N Feature26_N Feature27_N
## [31] Feature28_N  Feature29_N Feature30_N  Feature31_N Feature32_N Feature33_N
## [37] Feature34_N  Feature35_N Feature36_N  Feature37_N Feature38_N Feature39_N
## [43] Feature40_N  Feature41_N Feature42_N  Feature43_N Feature44_N Feature45_N
## [49] Feature46_N  Feature47_N Feature48_N  Feature49_N Feature50_N Feature51_N
## <0 rows> (or 0-length row.names)
```

Summary Statistics for the chemical Variables:

```
summary(cheese_df[, colnames(cheese_df)[4:54]])
```

```
##      Feature1_N      Feature2_N      Feature3_N      Feature4_N
## Min.   : 27475    Min.   : 1563290    Min.   : 241950    Min.   : 336.8
## 1st Qu.: 83139    1st Qu.: 9594022    1st Qu.: 983971    1st Qu.: 1368.0
## Median : 170654    Median : 12691200    Median : 2435900    Median : 5004.9
## Mean   : 1605011    Mean   : 18375077    Mean   : 3091465    Mean   : 12935.2
## 3rd Qu.: 1202990    3rd Qu.: 27187050    3rd Qu.: 4351233    3rd Qu.: 17427.8
## Max.   : 9039990    Max.   : 54837800    Max.   : 10260400    Max.   : 46521.3
##      Feature5_N      Feature6_N      Feature7_N      Feature8_N
## Min.   : 24622    Min.   : 50.53    Min.   : 837.8    Min.   : 24775
## 1st Qu.: 2895585    1st Qu.: 274.89    1st Qu.: 1415.0    1st Qu.: 32786
## Median : 7491520    Median : 659.87    Median : 1653.3    Median : 54024
## Mean   : 8531634    Mean   : 9697.33    Mean   : 1944.6    Mean   : 69069
## 3rd Qu.: 9130245    3rd Qu.: 2311.97    3rd Qu.: 2325.2    3rd Qu.: 77329
## Max.   : 23032500    Max.   : 61630.50    Max.   : 3417.4    Max.   : 221871
##      Feature9_N      Feature10_N      Feature11_N      Feature12_N
## Min.   : 267.2    Min.   : 16350    Min.   : 486.1    Min.   : 47.39
## 1st Qu.: 1175.0    1st Qu.: 24778    1st Qu.: 1583.4    1st Qu.: 102.78
## Median : 8973.5    Median : 30586    Median : 2707.5    Median : 155.44
## Mean   : 41768.9    Mean   : 29613    Mean   : 6331.7    Mean   : 231.76
## 3rd Qu.: 34970.6    3rd Qu.: 34841    3rd Qu.: 9974.2    3rd Qu.: 239.31
```

## Max. :215572.0	Max. :41768	Max. :26944.6	Max. :1229.91
## Feature13_N	Feature14_N	Feature15_N	Feature16_N
## Min. : 1.705	Min. : 16704	Min. : 854.4	Min. : 7745
## 1st Qu.: 48.910	1st Qu.: 37709	1st Qu.: 1182.9	1st Qu.: 27642
## Median :137.448	Median : 50029	Median : 2627.6	Median : 83203
## Mean :223.006	Mean : 51211	Mean : 8708.9	Mean :1148418
## 3rd Qu.:343.745	3rd Qu.: 61059	3rd Qu.: 7565.1	3rd Qu.: 472331
## Max. :651.362	Max. :106284	Max. :69980.9	Max. :6344840
## Feature17_N	Feature18_N	Feature19_N	Feature20_N
## Min. : 8.86	Min. : 4.206	Min. : 40749	Min. : 1784
## 1st Qu.: 360.60	1st Qu.: 711.966	1st Qu.: 89733	1st Qu.: 6445
## Median : 595.75	Median :1093.790	Median : 341059	Median : 14748
## Mean : 54618.85	Mean :1105.387	Mean :1457950	Mean : 26136
## 3rd Qu.: 1009.41	3rd Qu.:1424.350	3rd Qu.:1958983	3rd Qu.: 29721
## Max. :850093.00	Max. :3298.350	Max. :7986210	Max. :103269
## Feature21_N	Feature22_N	Feature23_N	Feature24_N
## Min. :4.959e+01	Min. :103259	Min. : 275.8	Min. : 7604
## 1st Qu.:2.351e+02	1st Qu.:150202	1st Qu.: 503.4	1st Qu.: 13842
## Median :6.150e+02	Median :172842	Median : 2668.2	Median : 17488
## Mean :8.122e+04	Mean :176278	Mean : 16335.4	Mean : 60145
## 3rd Qu.:1.861e+03	3rd Qu.:205475	3rd Qu.: 5701.1	3rd Qu.: 28566
## Max. :1.107e+06	Max. :271927	Max. :171371.0	Max. :408849
## Feature25_N	Feature26_N	Feature27_N	Feature28_N
## Min. : 7842	Min. : 13799	Min. : 163.9	Min. :2726
## 1st Qu.: 16476	1st Qu.: 29512	1st Qu.: 375.8	1st Qu.:3931
## Median : 138328	Median : 55657	Median : 784.2	Median :5112
## Mean : 283627	Mean : 85972	Mean : 5861.2	Mean :4844
## 3rd Qu.: 410716	3rd Qu.: 97994	3rd Qu.: 4577.9	3rd Qu.:5592
## Max. :1603290	Max. :363823	Max. :57593.0	Max. :7534
## Feature29_N	Feature30_N	Feature31_N	Feature32_N
## Min. : 9652	Min. : 3609	Min. : 2583	Min. :1406
## 1st Qu.:14112	1st Qu.: 5710	1st Qu.: 4089	1st Qu.:2044
## Median :18013	Median : 18930	Median : 5418	Median :2362
## Mean :18169	Mean : 358933	Mean : 7738	Mean :2322
## 3rd Qu.:19428	3rd Qu.: 485220	3rd Qu.: 6489	3rd Qu.:2649
## Max. :40617	Max. :2093160	Max. :25942	Max. :3236
## Feature33_N	Feature34_N	Feature35_N	Feature36_N
## Min. : 485.5	Min. : 5059	Min. : 333.5	Min. : 312.2
## 1st Qu.: 1026.6	1st Qu.: 106113	1st Qu.: 1063.0	1st Qu.: 514.5
## Median : 1726.5	Median : 293725	Median : 1434.3	Median : 751.0
## Mean : 4140.3	Mean : 516533	Mean : 54633.8	Mean : 2568.4
## 3rd Qu.: 5052.2	3rd Qu.: 548512	3rd Qu.: 22142.7	3rd Qu.: 1114.4
## Max. :25108.3	Max. :3368900	Max. :333969.0	Max. :14304.7
## Feature37_N	Feature38_N	Feature39_N	Feature40_N
## Min. : 651.8	Min. :21635	Min. : 32.44	Min. : 23.91
## 1st Qu.: 1656.2	1st Qu.:34525	1st Qu.: 132.25	1st Qu.: 116.60
## Median : 4450.2	Median :40183	Median : 1134.38	Median : 791.24
## Mean : 43220.0	Mean :46756	Mean : 8040.59	Mean : 3762.14
## 3rd Qu.: 10924.0	3rd Qu.:57140	3rd Qu.:10688.10	3rd Qu.: 3862.16
## Max. :528585.0	Max. :95030	Max. :40537.00	Max. :19563.90
## Feature41_N	Feature42_N	Feature43_N	Feature44_N
## Min. : 2696	Min. : 9.565	Min. :130.9	Min. : 251271
## 1st Qu.: 3696	1st Qu.: 36.148	1st Qu.:185.0	1st Qu.: 529736
## Median : 5133	Median : 510.546	Median :230.2	Median :2136560

```
## Mean : 8496 Mean :10825.397 Mean :269.2 Mean :2925054
## 3rd Qu.: 8368 3rd Qu.: 893.090 3rd Qu.:329.8 3rd Qu.:4584653
## Max. :29132 Max. :74194.200 Max. :568.8 Max. :9690890
## Feature45_N Feature46_N Feature47_N Feature48_N
## Min. : 1.715 Min. :7.044e+01 Min. :3.351e+00 Min. : 2629
## 1st Qu.: 58.085 1st Qu.:4.768e+02 1st Qu.:1.136e+01 1st Qu.: 11261
## Median : 555.904 Median :6.704e+03 Median :2.458e+01 Median : 48069
## Mean : 2359.117 Mean :1.245e+05 Mean :1.058e+04 Mean : 233893
## 3rd Qu.: 2098.927 3rd Qu.:1.157e+04 3rd Qu.:5.023e+01 3rd Qu.: 181878
## Max. :14231.400 Max. :1.583e+06 Max. :1.439e+05 Max. :1037710
## Feature49_N Feature50_N Feature51_N
## Min. : 790638 Min. : 835.8 Min. : 993.5
## 1st Qu.: 4472128 1st Qu.: 22372.4 1st Qu.:13699.5
## Median :10158450 Median : 70729.1 Median :33659.8
## Mean : 9404233 Mean : 93947.1 Mean :29616.4
## 3rd Qu.:13472450 3rd Qu.:157952.0 3rd Qu.:43322.1
## Max. :16010900 Max. :301481.0 Max. :71491.4
```

Checking variance for each column:

```
X <- cheese_df[, colnames(cheese_df)[4:54]]
log_X <- log(X)
vars <- sapply(X, var, na.rm = T)
vars
```

```
## Feature1_N Feature2_N Feature3_N Feature4_N Feature5_N Feature6_N
## 8.157573e+12 1.668490e+14 6.655245e+12 1.942932e+08 5.448053e+13 3.940476e+08
## Feature7_N Feature8_N Feature9_N Feature10_N Feature11_N Feature12_N
## 5.455542e+05 2.505700e+09 5.140714e+09 4.500682e+07 4.882418e+07 6.619300e+04
## Feature13_N Feature14_N Feature15_N Feature16_N Feature17_N Feature18_N
## 4.322348e+04 3.670724e+08 2.639424e+08 4.738388e+12 3.510902e+10 5.821587e+05
## Feature19_N Feature20_N Feature21_N Feature22_N Feature23_N Feature24_N
## 4.869282e+12 8.191700e+08 7.717626e+10 1.333881e+09 1.810323e+09 1.037633e+10
## Feature25_N Feature26_N Feature27_N Feature28_N Feature29_N Feature30_N
## 1.576232e+11 7.242049e+09 1.523196e+08 1.357369e+06 3.838431e+07 4.142197e+11
## Feature31_N Feature32_N Feature33_N Feature34_N Feature35_N Feature36_N
## 4.084727e+07 2.000984e+05 3.594367e+07 6.729793e+11 1.275979e+10 1.895492e+07
## Feature37_N Feature38_N Feature39_N Feature40_N Feature41_N Feature42_N
## 1.654121e+10 3.738347e+08 1.677893e+08 3.979082e+07 6.295021e+07 5.899237e+08
## Feature43_N Feature44_N Feature45_N Feature46_N Feature47_N Feature48_N
## 1.209173e+04 8.122371e+12 1.569734e+07 1.358740e+11 1.364603e+09 1.272227e+11
## Feature49_N Feature50_N Feature51_N
## 2.211770e+13 8.249023e+09 3.238783e+08
```

```
vars_log <- sapply(log_X, var, na.rm = T)
vars_log
```

```
## Feature1_N Feature2_N Feature3_N Feature4_N Feature5_N Feature6_N
## 3.52167113 0.70048811 1.12915823 2.28020075 3.43803939 4.65856821
## Feature7_N Feature8_N Feature9_N Feature10_N Feature11_N Feature12_N
## 0.13393474 0.37385366 4.05452652 0.06044013 1.14644529 0.53847511
## Feature13_N Feature14_N Feature15_N Feature16_N Feature17_N Feature18_N
## 2.36381519 0.15937207 1.56180712 5.05689543 5.95083216 4.01786392
## Feature19_N Feature20_N Feature21_N Feature22_N Feature23_N Feature24_N
## 2.77331258 1.23471823 6.30866226 0.04388309 3.02666764 1.25878509
## Feature25_N Feature26_N Feature27_N Feature28_N Feature29_N Feature30_N
```

```
## 3.02649539 0.73738310 2.70926516 0.06285497 0.08976753 5.08324700
## Feature31_N Feature32_N Feature33_N Feature34_N Feature35_N Feature36_N
## 0.38229825 0.04190679 1.09147994 2.54110634 5.07073829 1.30404258
## Feature37_N Feature38_N Feature39_N Feature40_N Feature41_N Feature42_N
## 2.83418376 0.14978326 5.46900053 4.24100179 0.52989575 7.70680676
## Feature43_N Feature44_N Feature45_N Feature46_N Feature47_N Feature48_N
## 0.14092147 1.43416973 5.43701735 7.08478727 6.72643841 3.72391171
## Feature49_N Feature50_N Feature51_N
## 0.64816038 2.98930499 0.96074320
```

Visually analyzing both raw and log-transformed distributions of chemicals:

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:stats':
##
##   filter, lag
##
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats 1.0.1 v readr 2.2.0
## v ggplot2 4.0.2 v stringr 1.6.0
## v lubridate 1.9.5 v tibble 3.3.1
## v purrr 1.2.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag() masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
chem_cols <- colnames(cheese_df)[4:54]
```

```
raw_long <- cheese_df |>
  select(PrimaryID, all_of(chem_cols)) |>
  pivot_longer(
    cols = all_of(chem_cols),
    names_to = "chemical",
    values_to = "value"
  ) |>
  mutate(scale = "Raw")
```

```
log_long <- cheese_df |>
  select(PrimaryID, all_of(chem_cols)) |>
  mutate(across(all_of(chem_cols), log)) |>
  pivot_longer(
    cols = all_of(chem_cols),
    names_to = "chemical",
    values_to = "value"
  ) |>
```

```

mutate(scale = "Log-transformed")

plot_long <- bind_rows(raw_long, log_long)
rbind(head(plot_long), tail(plot_long))

## # A tibble: 12 x 4
##   PrimaryID chemical      value scale
##   <chr>      <chr>      <dbl> <chr>
## 1 B1        Feature1_N    1163170 Raw
## 2 B1        Feature2_N    27168800 Raw
## 3 B1        Feature3_N    2094430 Raw
## 4 B1        Feature4_N      1347. Raw
## 5 B1        Feature5_N    2115440 Raw
## 6 B1        Feature6_N      50.5 Raw
## 7 G4        Feature46_N     6.20 Log-transformed
## 8 G4        Feature47_N     3.34 Log-transformed
## 9 G4        Feature48_N    13.7 Log-transformed
## 10 G4       Feature49_N    16.4 Log-transformed
## 11 G4       Feature50_N     7.74 Log-transformed
## 12 G4       Feature51_N    11.0 Log-transformed

library(ggplot2)
library(tidyverse)

plot_hist_density_long <- function(long_df, cols, bins = 20, ncol = 2) {

  plot_df <- long_df %>%
    filter(chemical %in% cols)

  if (nrow(plot_df) == 0) {
    stop("None of the supplied columns were found in long_df$chemical.")
  }

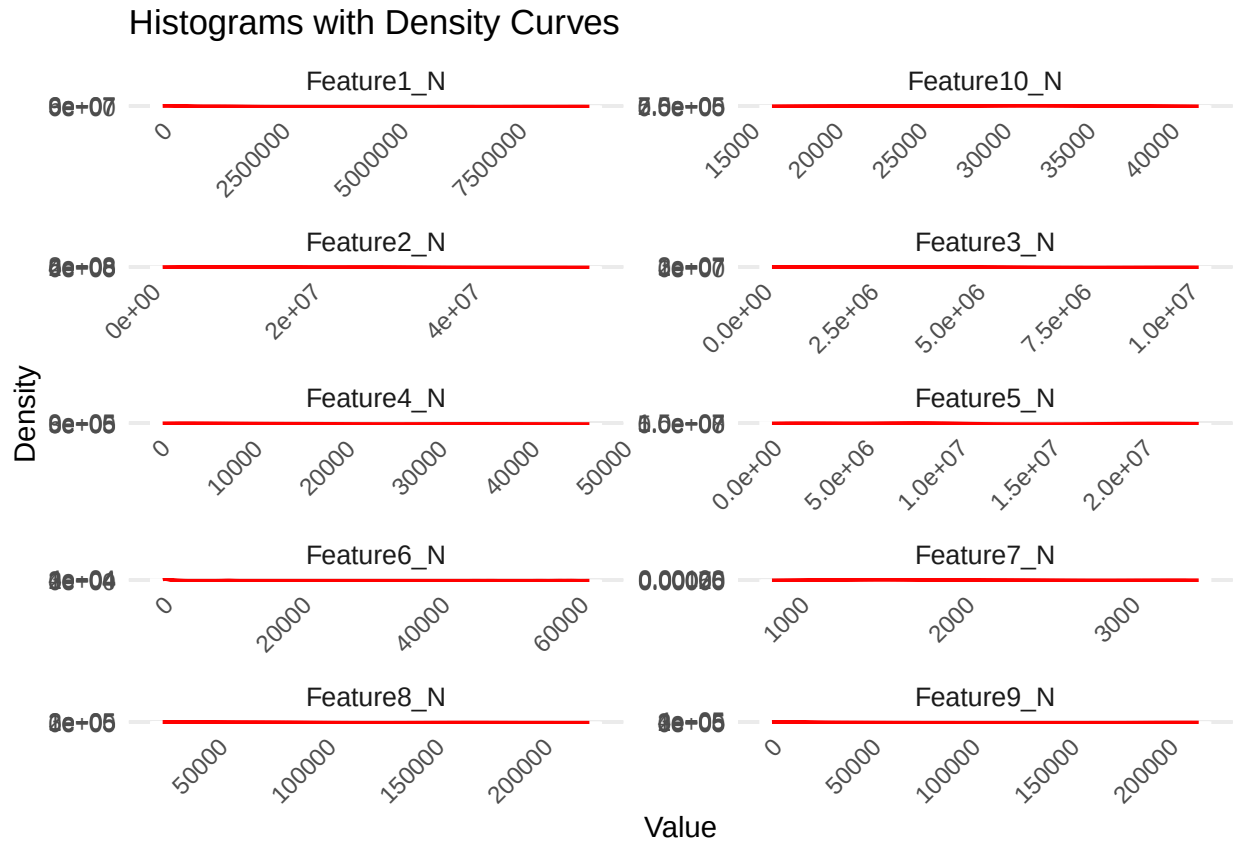
  ggplot(plot_df, aes(x = value)) +
    geom_histogram(aes(y = after_stat(density)),
      bins = bins,
      fill = "skyblue",
      color = "white",
      alpha = 0.7) +
    geom_density(color = "red", linewidth = 0.8, na.rm = TRUE) +
    facet_wrap(~ chemical, scales = "free", ncol = ncol) +
    theme_minimal() +
    labs(
      title = "Histograms with Density Curves",
      x = "Value",
      y = "Density"
    ) +
    theme(
      strip.text = element_text(size = 10),
      axis.text.x = element_text(angle = 45, hjust = 1)
    )
}

```

Features 1-10:

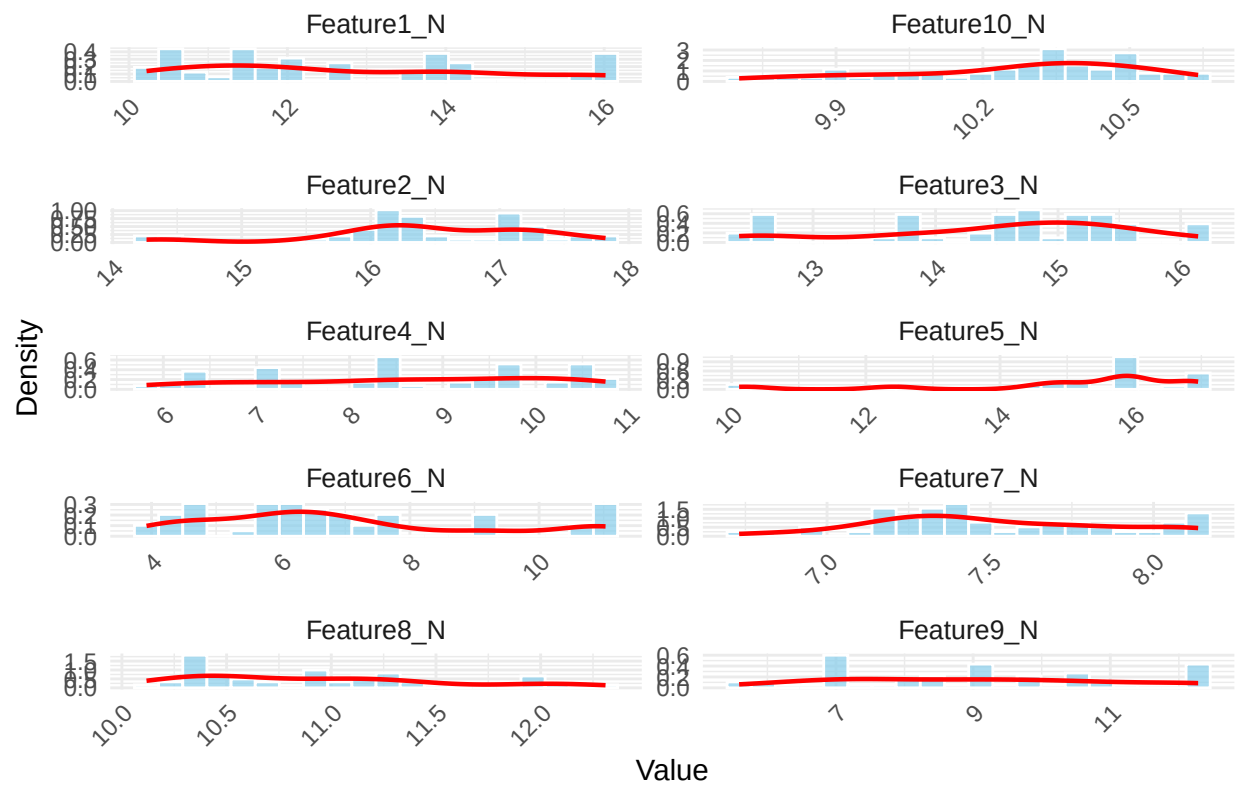
Raw Scale:

```
plot_hist_density_long(raw_long, chem_cols[1:10])
```



```
plot_hist_density_long(log_long, chem_cols[1:10])
```

Histograms with Density Curves

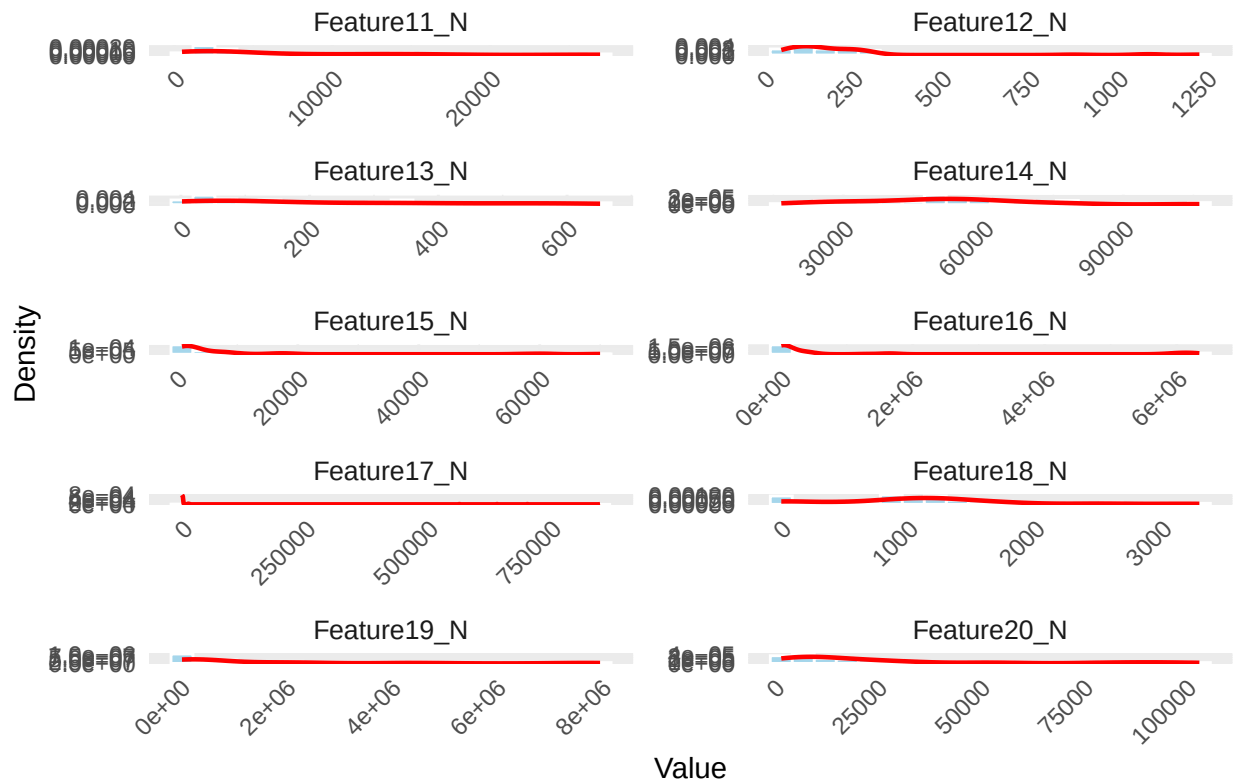


Features 11-20:

Raw Scale:

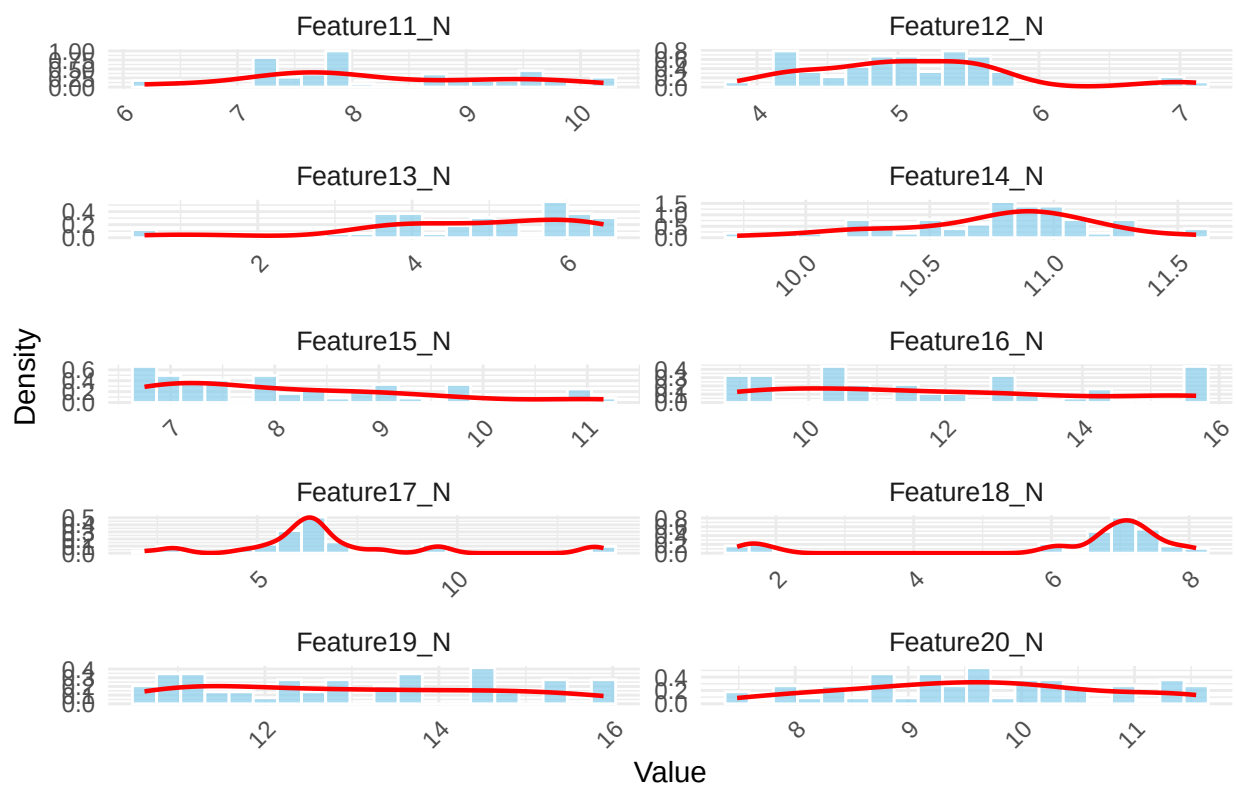
```
plot_hist_density_long(raw_long, chem_cols[11:20])
```


Histograms with Density Curves



```
plot_hist_density_long(log_long, chem_cols[11:20])
```

Histograms with Density Curves

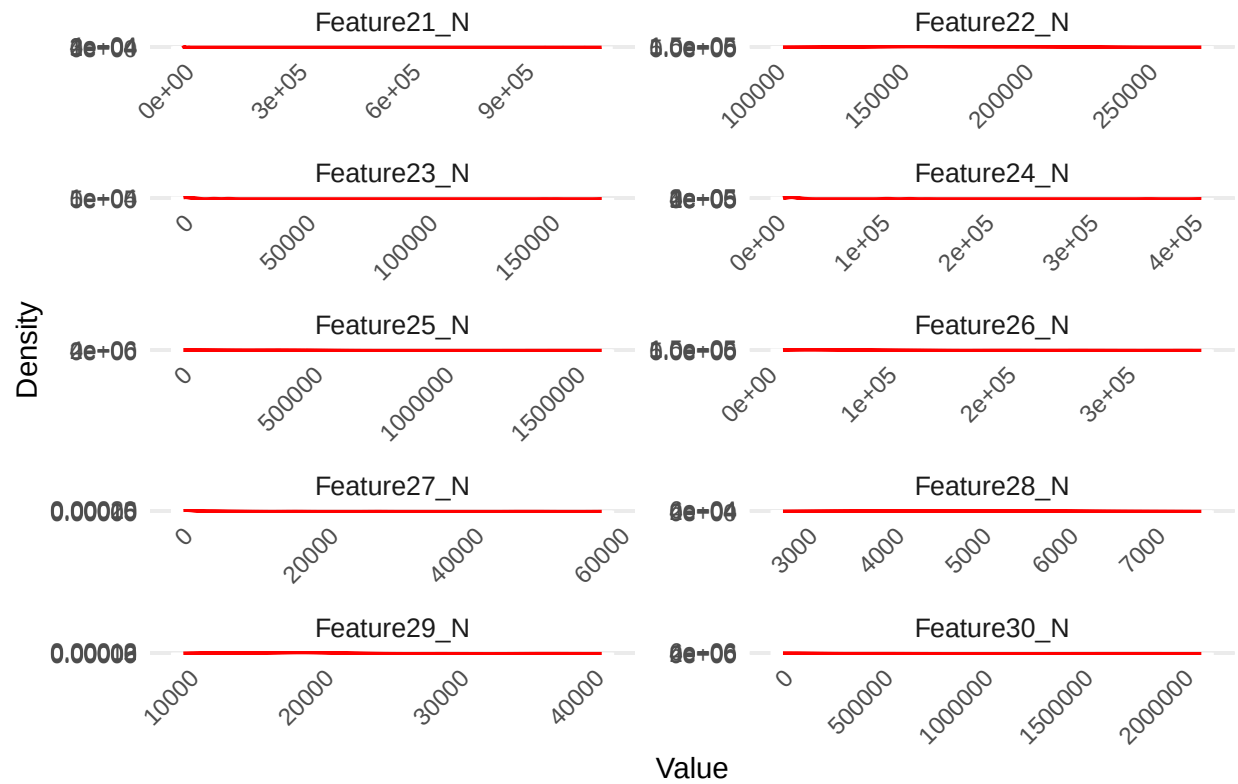


Features 21-30:

Raw Scale:

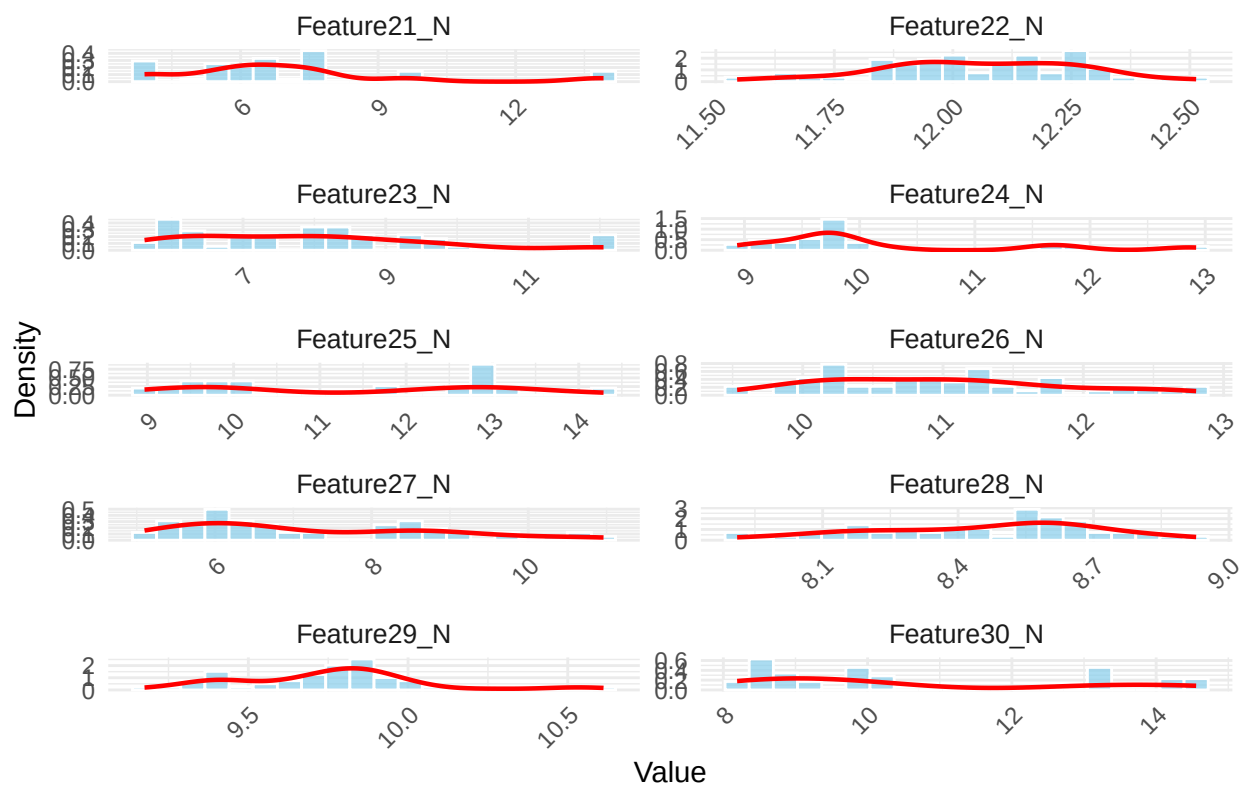
```
plot_hist_density_long(raw_long, chem_cols[21:30])
```

Histograms with Density Curves



```
plot_hist_density_long(log_long, chem_cols[21:30])
```

Histograms with Density Curves

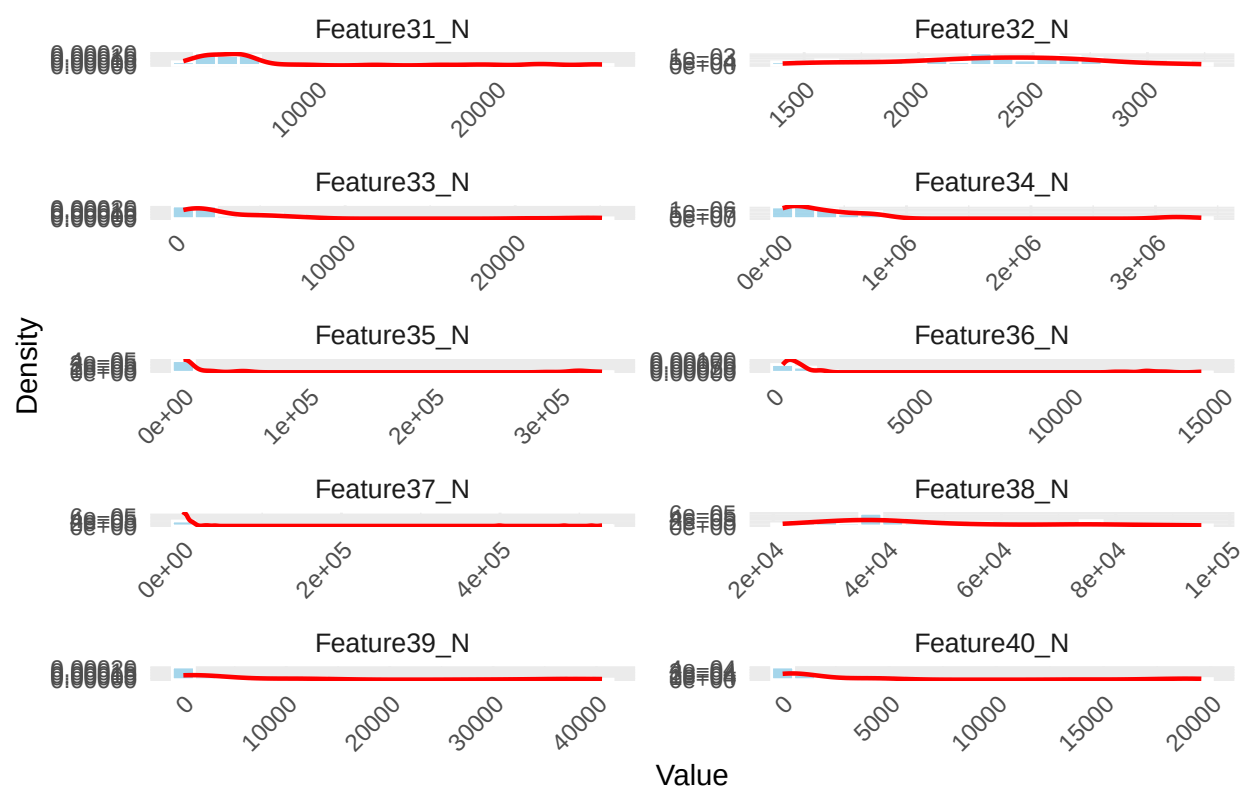


Features 31-40:

Raw Scale:

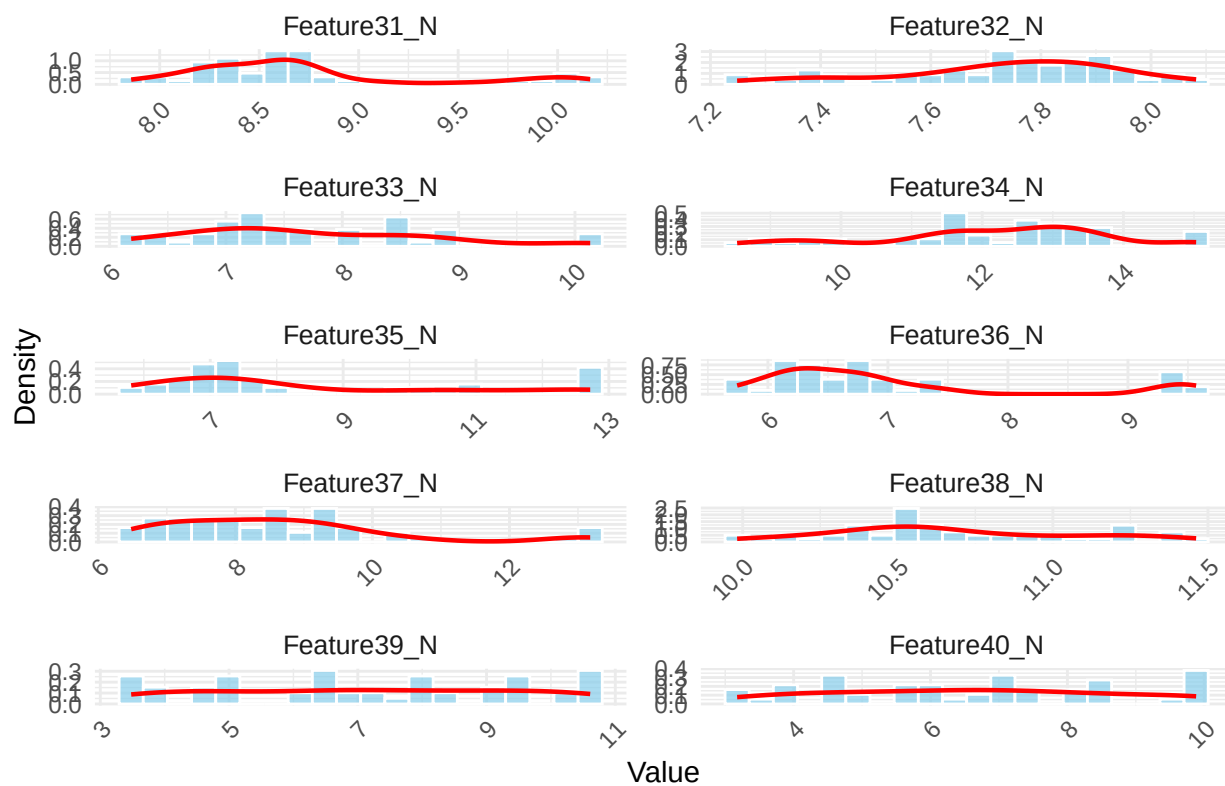
```
plot_hist_density_long(raw_long, chem_cols[31:40])
```

Histograms with Density Curves



```
plot_hist_density_long(log_long, chem_cols[31:40])
```

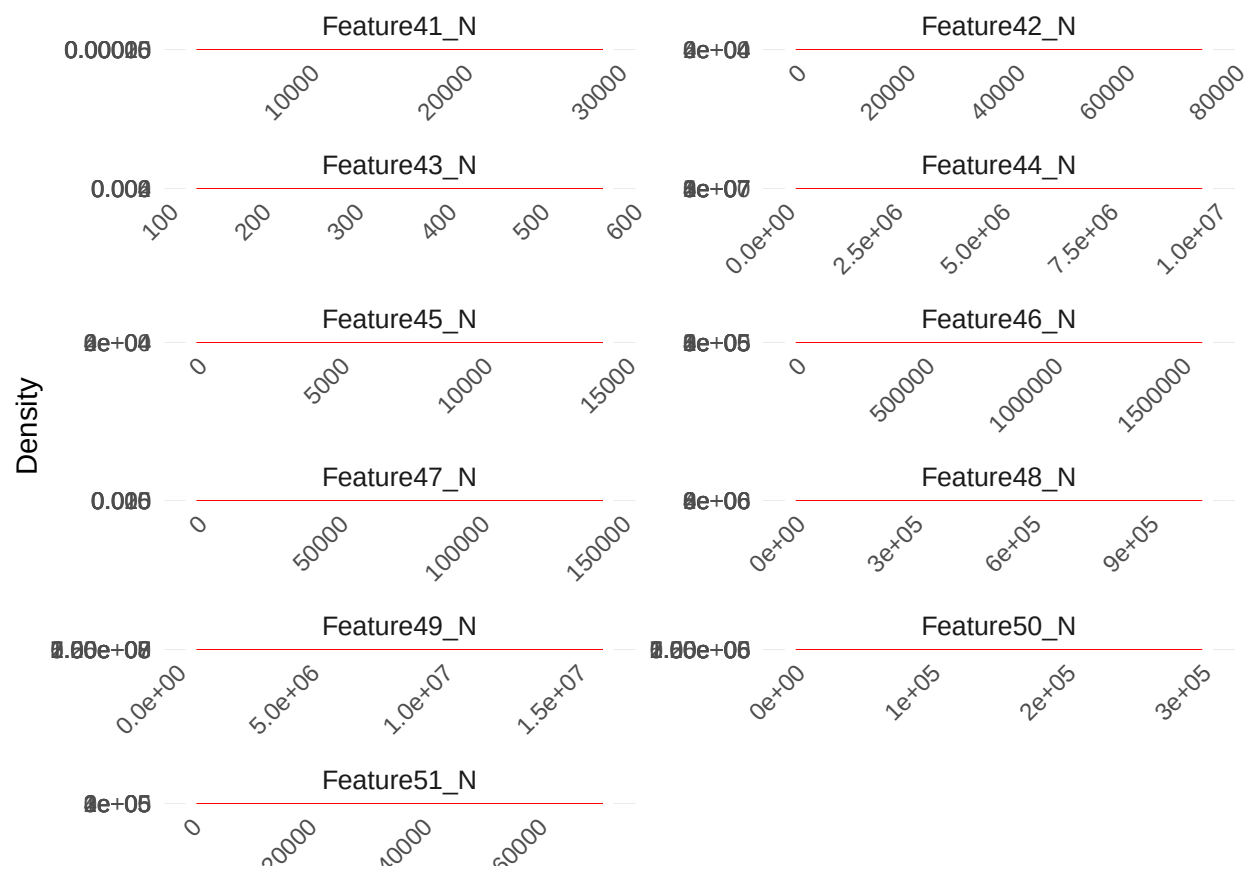
Histograms with Density Curves



Features 41-51:

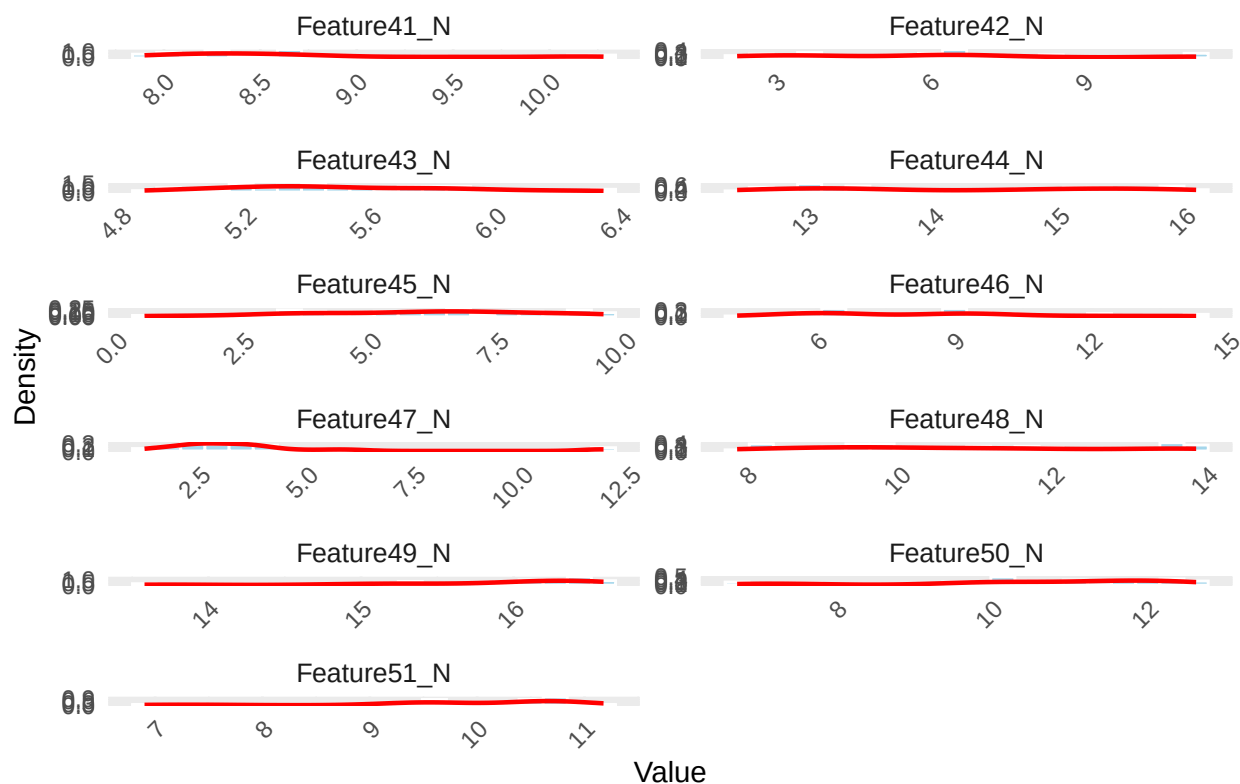
Raw Scale:

```
plot_hist_density_long(raw_long, chem_cols[41:51])
```



```
plot_hist_density_long(log_long, chem_cols[41:51])
```

Histograms with Density Curves



It seems reasonable to use the log-transformed variables, instead of the raw scale.

PCA on log-transformed chemical data:

```
pca_fit <- prcomp(log_X, center = TRUE, scale. = TRUE)
```

```
# Variance explained
```

```
eig <- pca_fit$sdev^2
```

```
var_explained <- eig / sum(eig)
```

```
cum_var_explained <- cumsum(var_explained)
```

```
pca_var_tbl <- tibble(
  PC = paste0("PC", seq_along(eig)),
  eigenvalue = eig,
  prop_var = var_explained,
  cum_prop_var = cum_var_explained
)
```

```
pca_var_tbl
```

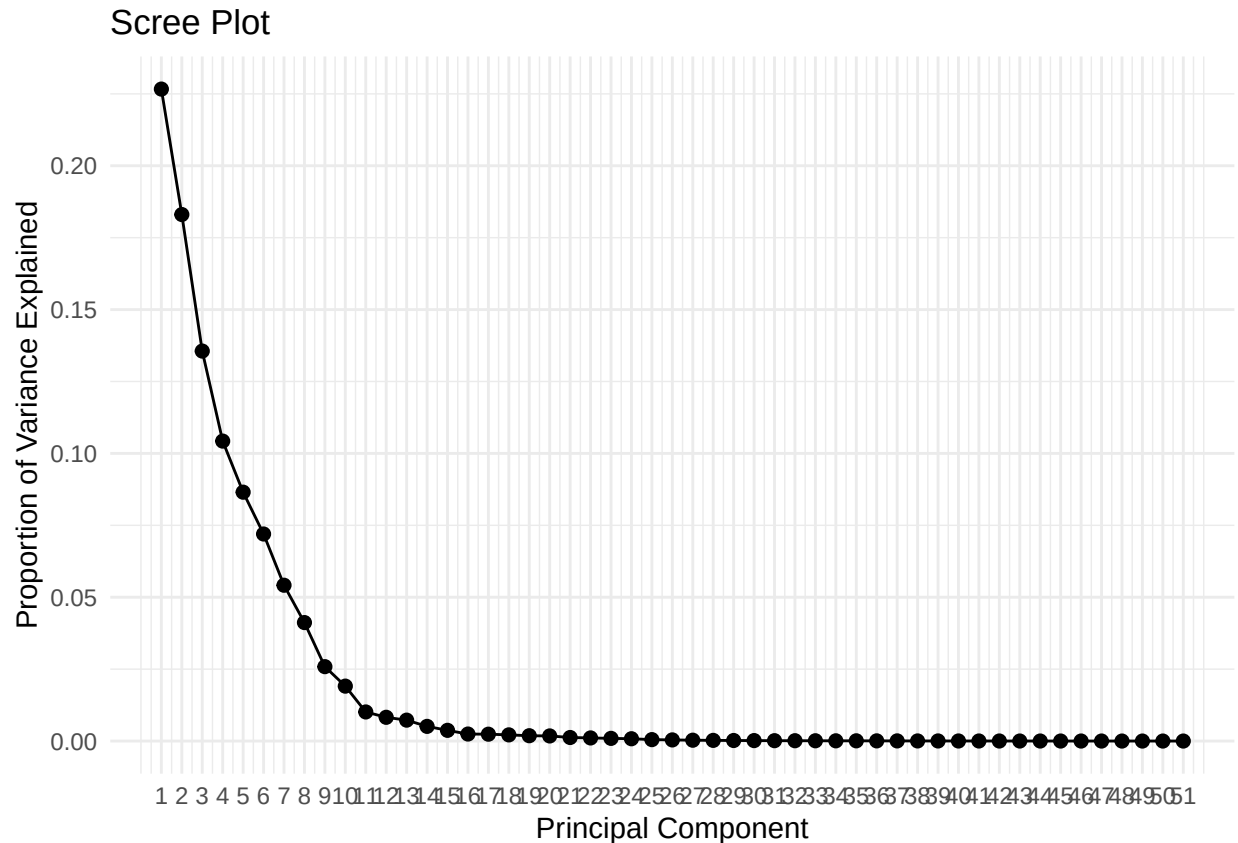
```
## # A tibble: 51 x 4
##   PC      eigenvalue prop_var cum_prop_var
##   <chr>      <dbl>    <dbl>      <dbl>
## 1 PC1        11.6      0.227      0.227
## 2 PC2         9.34      0.183      0.410
## 3 PC3         6.91      0.136      0.545
## 4 PC4         5.32      0.104      0.650
```



```
## 5 PC5      4.41    0.0865    0.736
## 6 PC6      3.67    0.0720    0.808
## 7 PC7      2.76    0.0542    0.862
## 8 PC8      2.10    0.0412    0.903
## 9 PC9      1.32    0.0259    0.929
## 10 PC10     0.974   0.0191    0.948
## # i 41 more rows
```

Scree plot:

```
ggplot(pca_var_tbl, aes(x = seq_along(PC), y = prop_var)) +
  geom_point(size = 2) +
  geom_line() +
  scale_x_continuous(breaks = seq_len(nrow(pca_var_tbl))) +
  labs(
    x = "Principal Component",
    y = "Proportion of Variance Explained",
    title = "Scree Plot"
  ) +
  theme_minimal()
```



Let's consider the first 8 PCs, based on the scree plot.

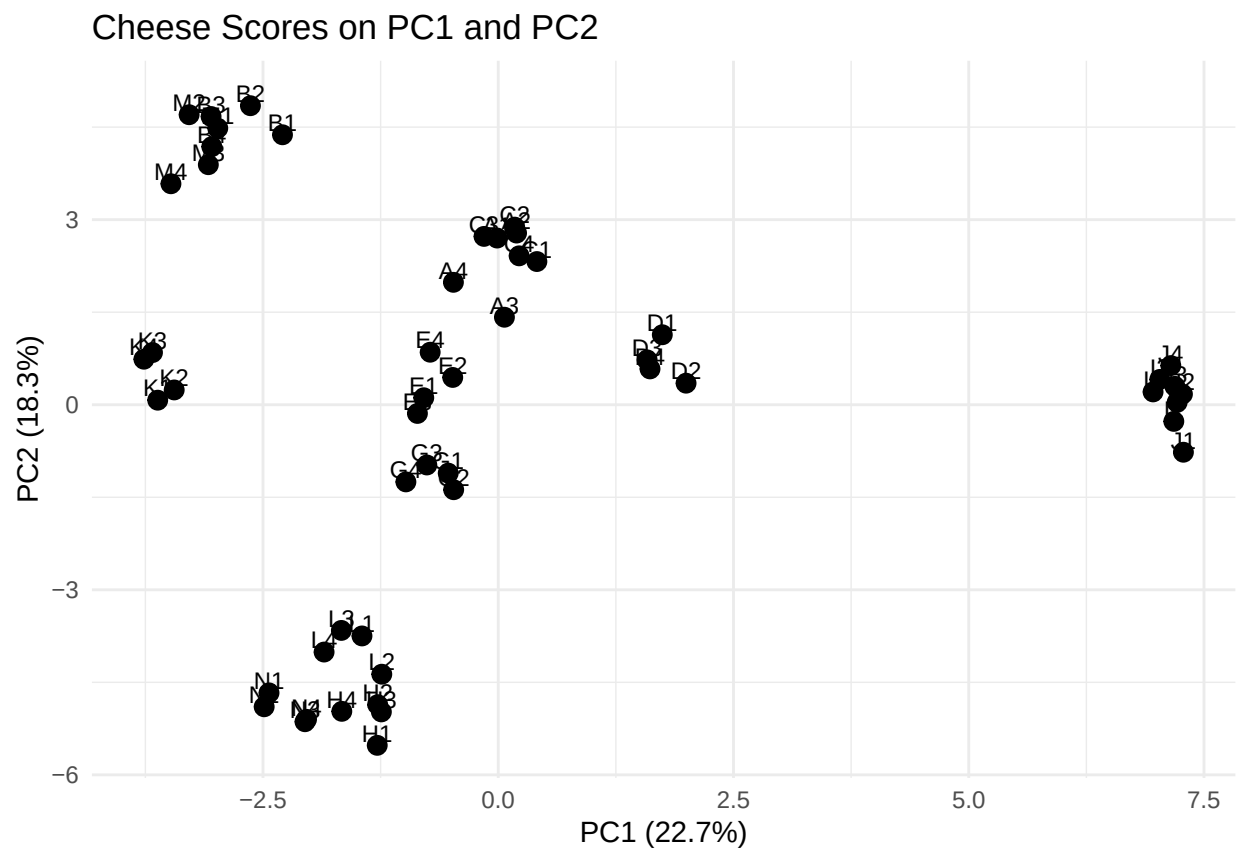
```
scores_df <- as_tibble(pca_fit$x) %>%
  mutate(PrimaryID = cheese_df$PrimaryID) %>%
  relocate(PrimaryID)

scores_df %>% select(PrimaryID, PC1, PC2, PC3, PC4, PC5, PC6, PC7, PC8)
```

```
## # A tibble: 52 x 9
##   PrimaryID PC1    PC2    PC3    PC4    PC5    PC6    PC7    PC8
##   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 B1      -2.29  4.38  -5.31 -2.24 -0.899  0.770 -1.75  1.49
## 2 B2      -2.63  4.85  -5.11 -2.25 -1.24   0.797 -2.67  1.57
## 3 B3      -3.05  4.67  -5.04 -2.05 -0.549  1.41  -2.75  1.74
## 4 B4      -3.04  4.18  -5.23 -1.97 -0.597  1.24  -2.25  1.61
## 5 H1      -1.29 -5.52  -1.87 -0.771 -1.78   1.42   1.23  0.733
## 6 H2      -1.28 -4.86  -1.27 -1.16 -1.14   1.17   0.0164 0.301
## 7 H3      -1.24 -4.98  -1.57 -0.846 -2.13   0.893  0.670  0.573
## 8 H4      -1.66 -4.97  -1.30 -0.258 -1.10   1.80   0.469  0.580
## 9 K1      -3.62  0.0738  4.49  1.54 -3.63  -2.71  -2.49 -0.521
## 10 K2     -3.44  0.240   5.12  0.891 -2.19  -1.63  -3.77 -0.437
## # i 42 more rows
```

Plotting the first 2 PCs:

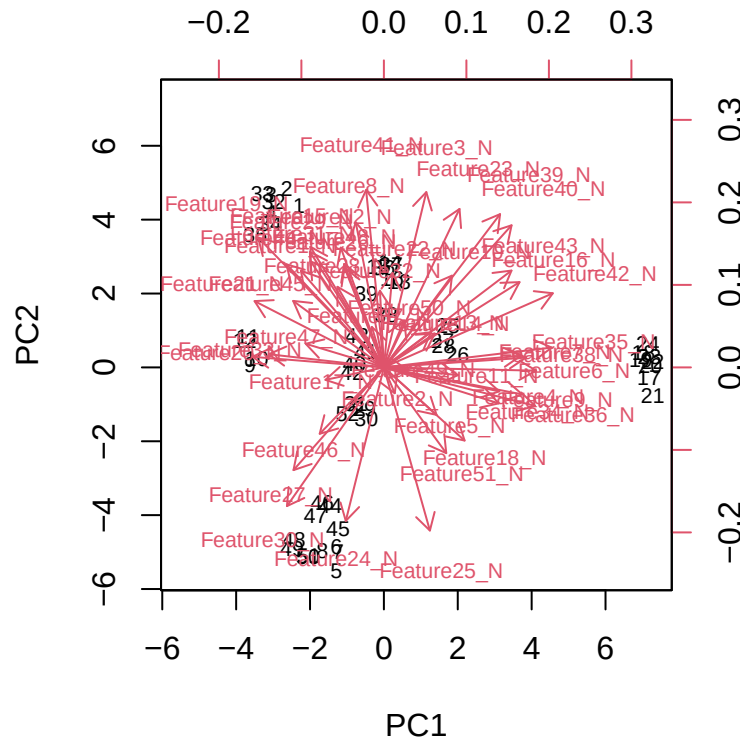
```
ggplot(scores_df, aes(x = PC1, y = PC2, label = PrimaryID)) +
  geom_point(size = 3) +
  geom_text(nudge_y = 0.2, size = 3) +
  labs(
    title = "Cheese Scores on PC1 and PC2",
    x = paste0("PC1 (", round(100 * pca_var_tbl$prop_var[1], 1), "%)"),
    y = paste0("PC2 (", round(100 * pca_var_tbl$prop_var[2], 1), "%)"),
  ) +
  theme_minimal()
```



It looks like the first 2 PCs do a good job of separating different cheese samples and are uncovering some patterns/trends.

Biplot:

```
biplot(pca_fit, scale = 0, cex = 0.7)
```



Interpretation: Use the biplot to find similar chemicals, long arrows mean they are important to the PC under study, same direction means positive relationship between chemicals, orthogonal arrows mean they have weak relationship.

We will extract the loadings to see which cheese's drive which PC:

```
loadings_df <- as_tibble(pca_fit$rotation, rownames = "PrimaryID")
```

```
loadings_df %>% select(PrimaryID, PC1, PC2, PC3, PC4, PC5, PC6, PC7, PC8)
```

```
## # A tibble: 51 x 9
##   PrimaryID    PC1    PC2    PC3    PC4    PC5    PC6    PC7    PC8
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Feature1~ -0.126  0.146  0.0727 -0.262 -0.0389 -0.189  0.0929 -0.108
## 2 Feature2~  0.0164 -0.0392 -0.100  -0.230  0.167  -0.228 -0.220  0.00473
## 3 Feature3~  0.0643  0.266  0.0576  0.0691  0.158  -0.0727  0.189  0.0142
## 4 Feature4~  0.175  -0.0363  0.0613 -0.0169 -0.0793 -0.0730  0.227  0.265
## 5 Feature5~  0.0798 -0.0719  0.155  -0.244  0.0969  0.146  -0.237 -0.153
## 6 Feature6~  0.231  -0.00495  0.161  0.0230  0.0186  0.0213  0.0946  0.260
## 7 Feature7~  0.207  0.0175 -0.0857 -0.197  -0.0711  0.0591 -0.206  0.0982
## 8 Feature8~ -0.0426  0.220  -0.110  0.0807 -0.0867 -0.182  -0.0576  0.322
## 9 Feature9~  0.210  -0.0409  0.117  0.195  0.159  -0.00271 -0.0438  0.0893
```

```
## 10 Feature1~ 0.103 0.139 -0.00397 0.0286 -0.322 0.0948 0.0684 -0.131
## # i 41 more rows
```

Based on the magnitude of the loadings we decide how strongly a chemical influences a PC and based on the sign we decide the direction of this relationship.

Function to find top n loadings:

```
get_top_loadings <- function(loadings_df, pc_name, top_n = 10) {
  loadings_df %>%
    transmute(
      PrimaryID,
      loading = .data[[pc_name]],
      abs_loading = abs(.data[[pc_name]])
    ) %>%
    arrange(desc(abs_loading)) %>%
    slice_head(n = top_n)
}
```

Top 10 chemicals for PC1:

```
top_pc1 <- get_top_loadings(loadings_df, "PC1")
top_pc1
```

```
## # A tibble: 10 x 3
##   PrimaryID loading abs_loading
##   <chr>      <dbl>      <dbl>
## 1 Feature42_N 0.256      0.256
## 2 Feature35_N 0.254      0.254
## 3 Feature6_N  0.231      0.231
## 4 Feature36_N 0.229      0.229
## 5 Feature38_N 0.215      0.215
## 6 Feature9_N  0.210      0.210
## 7 Feature7_N  0.207      0.207
## 8 Feature16_N 0.206      0.206
## 9 Feature20_N -0.199     0.199
## 10 Feature21_N -0.196     0.196
```

Top 10 chemicals for PC2:

```
top_pc2 <- get_top_loadings(loadings_df, "PC2")
top_pc2
```

```
## # A tibble: 10 x 3
##   PrimaryID loading abs_loading
##   <chr>      <dbl>      <dbl>
## 1 Feature41_N 0.269      0.269
## 2 Feature3_N  0.266      0.266
## 3 Feature25_N -0.247     0.247
## 4 Feature23_N 0.240      0.240
## 5 Feature24_N -0.233     0.233
## 6 Feature39_N 0.232      0.232
## 7 Feature8_N  0.220      0.220
## 8 Feature40_N 0.215      0.215
## 9 Feature30_N -0.210     0.210
## 10 Feature19_N 0.197      0.197
```

Top 10 chemicals for PC3:

```
top_pc3 <- get_top_loadings(loadings_df, "PC3")
top_pc3
```

```
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature49_N 0.321      0.321
## 2 Feature17_N 0.308      0.308
## 3 Feature47_N 0.256      0.256
## 4 Feature46_N 0.240      0.240
## 5 Feature18_N -0.240     0.240
## 6 Feature50_N 0.239      0.239
## 7 Feature37_N 0.219      0.219
## 8 Feature51_N 0.214      0.214
## 9 Feature31_N 0.198      0.198
## 10 Feature16_N 0.185      0.185
```

Top 10 chemicals for PC4:

```
top_pc4 <- get_top_loadings(loadings_df, "PC4")
top_pc4
```

```
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature13_N -0.312     0.312
## 2 Feature11_N -0.287     0.287
## 3 Feature45_N -0.264     0.264
## 4 Feature1_N  -0.262     0.262
## 5 Feature5_N  -0.244     0.244
## 6 Feature2_N  -0.230     0.230
## 7 Feature50_N -0.228     0.228
## 8 Feature26_N -0.221     0.221
## 9 Feature51_N 0.198      0.198
## 10 Feature18_N -0.198     0.198
```

Top 10 chemicals for PC5:

```
get_top_loadings(loadings_df, "PC5")
```

```
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature10_N -0.322     0.322
## 2 Feature14_N -0.286     0.286
## 3 Feature44_N 0.276      0.276
## 4 Feature48_N 0.260      0.260
## 5 Feature37_N -0.249     0.249
## 6 Feature32_N 0.227      0.227
## 7 Feature28_N 0.217      0.217
## 8 Feature21_N -0.209     0.209
## 9 Feature12_N -0.196     0.196
## 10 Feature2_N 0.167      0.167
```

Top 10 chemicals for PC6:

```
get_top_loadings(loadings_df, "PC6")
```

```
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature26_N -0.289     0.289
## 2 Feature33_N  0.243     0.243
## 3 Feature34_N -0.240     0.240
## 4 Feature22_N -0.239     0.239
## 5 Feature38_N -0.237     0.237
## 6 Feature29_N  0.237     0.237
## 7 Feature14_N -0.228     0.228
## 8 Feature2_N   -0.228     0.228
## 9 Feature46_N -0.222     0.222
## 10 Feature48_N -0.208     0.208
```

Top 10 chemicals for PC7:

```
get_top_loadings(loadings_df, "PC7")
```

```
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature12_N  0.294     0.294
## 2 Feature48_N -0.264     0.264
## 3 Feature37_N -0.249     0.249
## 4 Feature5_N   -0.237     0.237
## 5 Feature4_N    0.227     0.227
## 6 Feature2_N   -0.220     0.220
## 7 Feature21_N -0.217     0.217
## 8 Feature50_N  0.216     0.216
## 9 Feature32_N -0.213     0.213
## 10 Feature47_N -0.212     0.212
```

Top 10 chemicals for PC8:

```
get_top_loadings(loadings_df, "PC8")
```

```
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature27_N  0.392     0.392
## 2 Feature8_N   0.322     0.322
## 3 Feature15_N  0.273     0.273
## 4 Feature4_N   0.265     0.265
## 5 Feature6_N   0.260     0.260
## 6 Feature35_N  0.247     0.247
## 7 Feature49_N -0.240     0.240
## 8 Feature20_N  0.213     0.213
## 9 Feature25_N  0.188     0.188
## 10 Feature34_N -0.183     0.183
```

No single chemical loads very strongly on any PC, there's a lot of cross loading and shared variance.

```
pc_summary <- list(
  PC1 = top_pc1,
  PC2 = top_pc2,
```

```

PC3 = top_pc3,
PC4 = top_pc4
)

```

```
pc_summary
```

```

## $PC1
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature42_N 0.256      0.256
## 2 Feature35_N 0.254      0.254
## 3 Feature6_N  0.231      0.231
## 4 Feature36_N 0.229      0.229
## 5 Feature38_N 0.215      0.215
## 6 Feature9_N  0.210      0.210
## 7 Feature7_N  0.207      0.207
## 8 Feature16_N 0.206      0.206
## 9 Feature20_N -0.199     0.199
## 10 Feature21_N -0.196     0.196
##
## $PC2
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature41_N 0.269      0.269
## 2 Feature3_N  0.266      0.266
## 3 Feature25_N -0.247     0.247
## 4 Feature23_N 0.240      0.240
## 5 Feature24_N -0.233     0.233
## 6 Feature39_N 0.232      0.232
## 7 Feature8_N  0.220      0.220
## 8 Feature40_N 0.215      0.215
## 9 Feature30_N -0.210     0.210
## 10 Feature19_N 0.197      0.197
##
## $PC3
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading
##   <chr>       <dbl>     <dbl>
## 1 Feature49_N 0.321      0.321
## 2 Feature17_N 0.308      0.308
## 3 Feature47_N 0.256      0.256
## 4 Feature46_N 0.240      0.240
## 5 Feature18_N -0.240     0.240
## 6 Feature50_N 0.239      0.239
## 7 Feature37_N 0.219      0.219
## 8 Feature51_N 0.214      0.214
## 9 Feature31_N 0.198      0.198
## 10 Feature16_N 0.185      0.185
##
## $PC4
## # A tibble: 10 x 3
##   PrimaryID   loading abs_loading

```

```
##      <chr>      <dbl>      <dbl>
##  1 Feature13_N -0.312      0.312
##  2 Feature11_N -0.287      0.287
##  3 Feature45_N -0.264      0.264
##  4 Feature1_N  -0.262      0.262
##  5 Feature5_N  -0.244      0.244
##  6 Feature2_N  -0.230      0.230
##  7 Feature50_N -0.228      0.228
##  8 Feature26_N -0.221      0.221
##  9 Feature51_N  0.198      0.198
## 10 Feature18_N -0.198      0.198
```

The first 4 PCs explain 65% of the variability and seem reasonable, we will retain those, but we still have upto 8 PCs we might use them to uncover some patterns after clustering.

To validate the PCA and check if the patterns (chemicals that load strongly on the first 4 PCs) actually have some pattern of going together we will cluster the chemicals to find chemicals that are similar or close to each other based on a correlation-based distance metric, so our vector will represent a chemical and the elements of the vector represent the different cheeses.

Heirarchical Clustering:

```
log_X_scaled <- scale(log_X, center = T, scale = T)
chem_mat <- t(log_X_scaled)
dim(chem_mat)
```

```
## [1] 51 52
```

For distance metric we will use 1-r, this computes similarity between chemicals based on how they vary across cheeses,

```
chem_cor <- cor(t(chem_mat), use = "pairwise.complete.obs")
chem_dist <- as.dist(1 - abs(chem_cor))
```

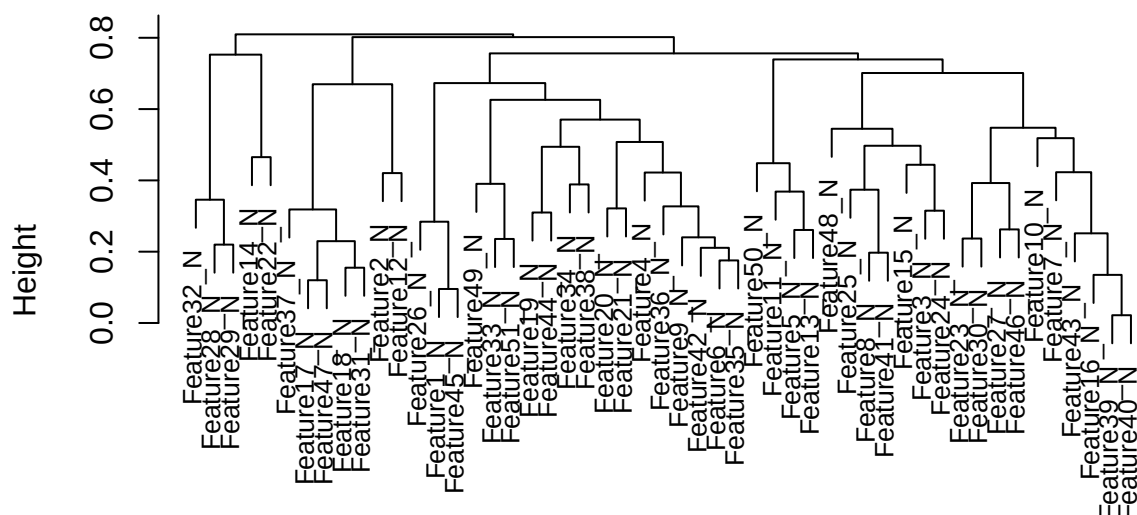
This metric does not care about the direction only the strength of the relationship.

Dendogram:

```
hc_chem <- hclust(chem_dist, method = "average")

plot(hc_chem, main = "Hierarchical Clustering of Chemicals",
     xlab = "", sub = "", cex = 0.8)
```


Hierarchical Clustering of Chemicals



I think having 6 clusters seem reasonable.

```
k <- 4
chem_clusters <- cutree(hc_chem, k = k)

cluster_df <- tibble(
  chemical = names(chem_clusters),
  cluster = chem_clusters
) %>%
  arrange(cluster)

#cluster_df
cluster_df %>% group_by(as.factor(cluster)) %>% summarise(n = n())
```

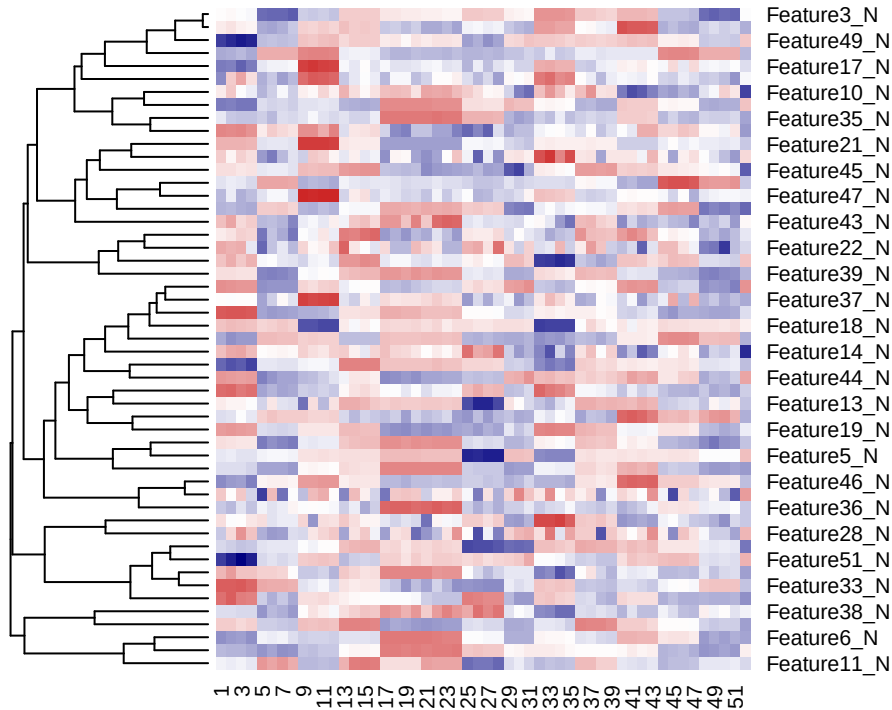
```
## # A tibble: 4 x 2
##   `as.factor(cluster)`      n
##   <fct>                <int>
## 1 1                      18
## 2 2                       7
## 3 3                      21
## 4 4                       5
```

Let's look at a heatmap with the clusters:

```
ordered_chemicals <- hc_chem$labels[hc_chem$order]

heatmap_mat <- chem_mat[ordered_chemicals, ,drop = FALSE]
```

```
heatmap(heatmap_mat,
  Rowv = as.dendrogram(hc_chem),
  Colv = NA,
  scale = "none",
  margins = c(6, 10),
  col = colorRampPalette(c("navy", "white", "firebrick3"))(100))
```



Validating results, does 4 clusters make sense:

```
library(cluster)
k <- 5
cluster_assign <- cutree(hc_chem, k = k)

sil <- silhouette(cluster_assign, chem_dist)

summary(sil)
```

```
## Silhouette of 51 units in 5 clusters from silhouette.default(x = cluster_assign, dist = chem_dist) :
## Cluster sizes and average silhouette widths:
##      18      7      21      2      3
## 0.17675303 0.34991311 0.08202038 0.32598965 0.56973988
## Individual silhouette widths:
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.19417  0.07878  0.18971  0.19048  0.28548  0.63781

mean_sil <- mean(sil[, "sil_width"])
mean_sil
```

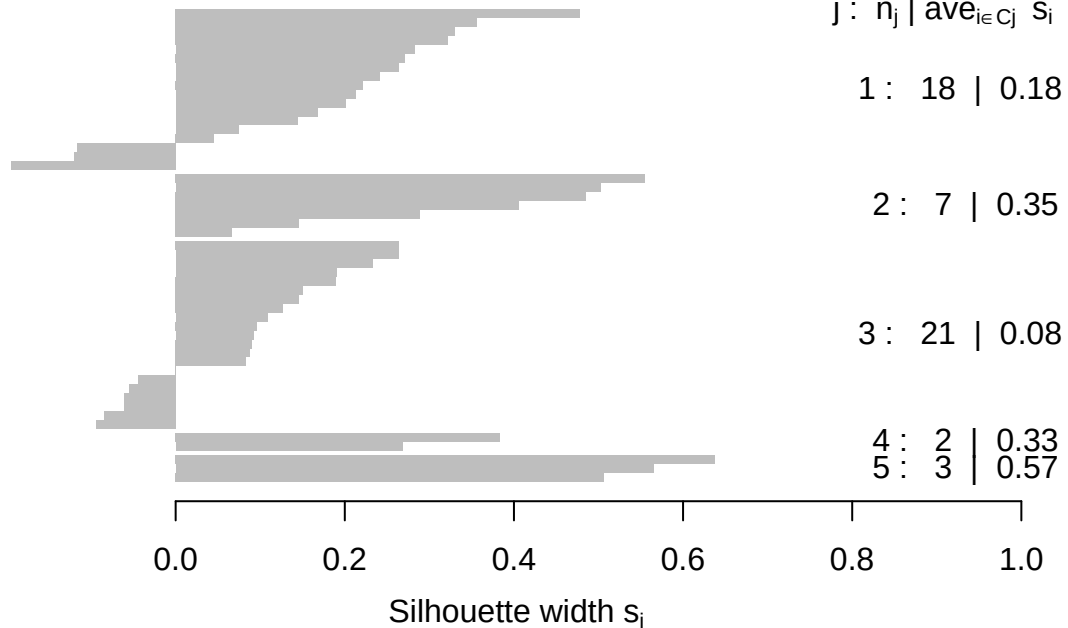
```
## [1] 0.1904818
```

silhouette plot:

```
plot(sil, border = NA, main = "Silhouette Plot for k = 4")
```

Silhouette Plot for k = 4

n = 51



Average silhouette width : 0.19

We would like avg Silhouette widths to be > 0.25 for each cluster

```
sil_df <- as.data.frame(sil[, 1:3]) %>%  
  rownames_to_column("chemical") %>%  
  as_tibble() %>%  
  rename(  
    cluster = cluster,  
    neighbor = neighbor,  
    sil_width = sil_width  
  ) %>%  
  arrange(cluster, desc(sil_width))
```

```
sil_df[sil_df$sil_width<0.2,]
```

```
## # A tibble: 27 x 4  
##   chemical cluster neighbor sil_width  
##   <chr>      <dbl>    <dbl>    <dbl>  
## 1 4          1         5     0.168  
## 2 34         1         5     0.144  
## 3 26         1         3     0.0744  
## 4 38         1         4     0.0456  
## 5 21         1         2    -0.116  
## 6 49         1         2    -0.120
```

```
## 7 44      1      5 -0.194
## 8 31      2      5  0.145
## 9 12      2      3  0.0665
## 10 43     3      4  0.190
## # i 17 more rows
```

Cluster-level averages:

```
sil_cluster_summary <- sil_df %>%
  group_by(cluster) %>%
  summarise(
    n = n(),
    avg_sil_width = mean(sil_width),
    min_sil_width = min(sil_width),
    n_negative = sum(sil_width < 0)
  )
```

```
sil_cluster_summary
```

```
## # A tibble: 5 x 5
##   cluster      n avg_sil_width min_sil_width n_negative
##   <dbl> <int>      <dbl>      <dbl>      <int>
## 1     1     18      0.177      -0.194         3
## 2     2      7      0.350       0.0665         0
## 3     3     21      0.0820     -0.0939         6
## 4     4      2      0.326       0.268         0
## 5     5      3      0.570       0.506         0
```

Cluster 3 is kind of alarming when $k = 5$.

Finding optimal number of clusters:

```
compare_k <- function(hc_obj, dist_obj, k_values = 2:8) {
  map_dfr(k_values, function(k) {
    cl <- cutree(hc_obj, k = k)
    sil <- silhouette(cl, dist_obj)

    tibble(
      k = k,
      avg_silhouette = mean(sil[, "sil_width"]),
      min_cluster_size = min(table(cl)),
      max_cluster_size = max(table(cl)),
      n_negative = sum(sil[, "sil_width"] < 0)
    )
  })
}
```

```
k_summary <- compare_k(hc_chem, chem_dist, k_values = 2:8)
k_summary
```

```
## # A tibble: 7 x 5
##   k avg_silhouette min_cluster_size max_cluster_size n_negative
##   <int>      <dbl>      <int>      <int>      <int>
## 1     2      0.133         5         46         4
## 2     3      0.161         5         39         5
## 3     4      0.203         5         21         6
## 4     5      0.190         2         21         9
```

## 5	6	0.215	2	18	9
## 6	7	0.241	2	18	8
## 7	8	0.246	2	15	10

None of these k's do a very good job of clustering, so we'll use it as an exploratory tool. I will go ahead with k = 4, it seems reasonable and the tradeoff is fair.

I am going forward with 4 clusters since that avoids clusters with too few features, the next step is to compare whether they load on the same PC, i.e. whether features in the same cluster load strongly to the same PC.

Comparing the 4 clusters with the 4 PCs:

```
k <- 4

cluster_assign <- cutree(hc_chem, k = k)

loadings_df <- as.data.frame(pca_fit$rotation[, 1:4]) %>%
  rownames_to_column("chemical") %>%
  as_tibble()

# cluster table
cluster_df <- tibble(
  chemical = names(cluster_assign),
  cluster = as.integer(cluster_assign)
)

# merge and compute dominant PC info
compare_df <- cluster_df %>%
  left_join(loadings_df, by = "chemical") %>%
  rowwise() %>%
  mutate(
    dominant_PC = c("PC1", "PC2", "PC3", "PC4")[which.max(abs(c(PC1, PC2, PC3, PC4)))],
    dominant_loading = c(PC1, PC2, PC3, PC4)[which.max(abs(c(PC1, PC2, PC3, PC4)))],
    loading_sign = ifelse(dominant_loading >= 0, "Positive", "Negative"),
    max_abs_loading = max(abs(c(PC1, PC2, PC3, PC4)))
  ) %>%
  ungroup() %>%
  arrange(cluster, dominant_PC, desc(max_abs_loading))

compare_df
```

```
## # A tibble: 51 x 10
##   chemical    cluster    PC1    PC2    PC3    PC4 dominant_PC
##   <chr>      <int> <dbl> <dbl> <dbl> <dbl> <chr>
## 1 Feature42_N      1  0.256  0.112  0.0585  0.110  PC1
## 2 Feature35_N      1  0.254  0.0300  0.0459  0.0625  PC1
## 3 Feature6_N       1  0.231 -0.00495  0.161  0.0230  PC1
## 4 Feature36_N      1  0.229 -0.0584  0.103  0.0181  PC1
## 5 Feature38_N      1  0.215  0.0148 -0.0488  0.0369  PC1
## 6 Feature9_N       1  0.210 -0.0409  0.117  0.195  PC1
## 7 Feature20_N      1 -0.199  0.0161  0.0545 -0.177  PC1
## 8 Feature21_N      1 -0.196  0.100  0.0771 -0.00962 PC1
## 9 Feature4_N       1  0.175 -0.0363  0.0613 -0.0169  PC1
## 10 Feature33_N     1 -0.174  0.0200 -0.136 -0.126  PC1
## # i 41 more rows
## # i 3 more variables: dominant_loading <dbl>, loading_sign <chr>,
```

```
## # max_abs_loading <dbl>
```

Checking whether each cluster is mostly associated with 1 PC:

```
cluster_pc_summary <- compare_df %>%  
  count(cluster, dominant_PC, loading_sign) %>%  
  group_by(cluster) %>%  
  mutate(prop = n / sum(n)) %>%  
  arrange(cluster, desc(prop))
```

```
cluster_pc_summary
```

```
## # A tibble: 16 x 5  
## # Groups:   cluster [4]  
##   cluster dominant_PC loading_sign      n  prop  
##   <int> <chr>      <chr>    <int> <dbl>  
## 1      1 PC1      Positive      8 0.444  
## 2      1 PC1      Negative      3 0.167  
## 3      1 PC4      Negative      3 0.167  
## 4      1 PC2      Positive      2 0.111  
## 5      1 PC3      Positive      2 0.111  
## 6      2 PC3      Positive      4 0.571  
## 7      2 PC2      Positive      1 0.143  
## 8      2 PC3      Negative      1 0.143  
## 9      2 PC4      Negative      1 0.143  
## 10     3 PC2      Positive      9 0.429  
## 11     3 PC2      Negative      4 0.190  
## 12     3 PC1      Positive      3 0.143  
## 13     3 PC4      Negative      3 0.143  
## 14     3 PC3      Positive      2 0.0952  
## 15     4 PC2      Positive      4 0.8  
## 16     4 PC3      Negative      1 0.2
```

Average loading patterns by cluster:

```
cluster_loading_summary <- compare_df %>%  
  group_by(cluster) %>%  
  summarise(  
    n_features = n(),  
    mean_abs_PC1 = mean(abs(PC1)),  
    mean_abs_PC2 = mean(abs(PC2)),  
    mean_abs_PC3 = mean(abs(PC3)),  
    mean_abs_PC4 = mean(abs(PC4))  
  )
```

```
cluster_loading_summary
```

```
## # A tibble: 4 x 6  
##   cluster n_features mean_abs_PC1 mean_abs_PC2 mean_abs_PC3 mean_abs_PC4  
##   <int>     <int>      <dbl>      <dbl>      <dbl>      <dbl>  
## 1      1         18      0.172      0.0764     0.0999     0.114  
## 2      2          7      0.0772     0.0869     0.189      0.148  
## 3      3         21      0.110      0.160      0.114      0.120  
## 4      4          5      0.0557     0.122      0.0974     0.0568
```

Whether chemicals within same cluster have similar PCA loading patterns:

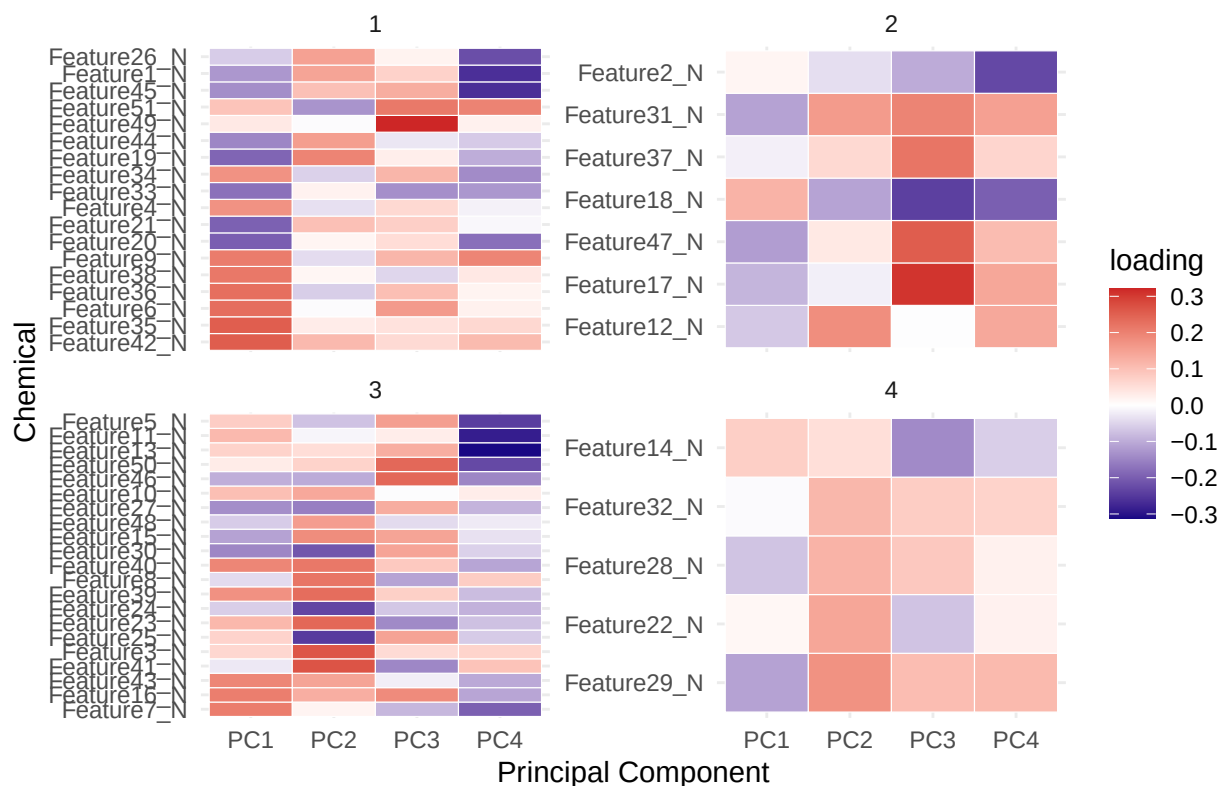
```

plot_df <- compare_df %>%
  select(chemical, cluster, PC1, PC2, PC3, PC4) %>%
  pivot_longer(
    cols = starts_with("PC"),
    names_to = "PC",
    values_to = "loading"
  ) %>%
  mutate(
    chemical = factor(
      chemical,
      levels = compare_df %>% arrange(cluster) %>% pull(chemical)
    )
  )

ggplot(plot_df, aes(x = PC, y = chemical, fill = loading)) +
  geom_tile(color = "white") +
  facet_wrap(~ cluster, scales = "free_y", ncol = 2) +
  scale_fill_gradient2(
    low = "navy",
    mid = "white",
    high = "firebrick3",
    midpoint = 0
  ) +
  theme_minimal() +
  labs(
    title = "PCA Loadings by Chemical Cluster",
    x = "Principal Component",
    y = "Chemical"
  )

```

PCA Loadings by Chemical Cluster



We don't have a very good structure from the PCA and clustering but it's acceptable, we will use these later on to validate the results of our analysis, we will save the results from our analysis and move to screening the most important chemicals.

```
k_clusters <- 4
cluster_assign <- cutree(hc_chem, k = k_clusters)

chemical_annotation <- as.data.frame(pca_fit$rotation[, 1:4]) %>%
  rownames_to_column("chemical") %>%
  as_tibble() %>%
  rename(
    PC1_loading = PC1,
    PC2_loading = PC2,
    PC3_loading = PC3,
    PC4_loading = PC4
  ) %>%
  rowwise() %>%
  mutate(
    cluster_4 = unname(cluster_assign[chemical]),
    dominant_PC = c("PC1", "PC2", "PC3", "PC4")[which.max(abs(c(PC1_loading, PC2_loading, PC3_loading, PC4_loading)))],
    dominant_loading = c(PC1_loading, PC2_loading, PC3_loading, PC4_loading)[which.max(abs(c(PC1_loading, PC2_loading, PC3_loading, PC4_loading)))],
    loading_sign = ifelse(dominant_loading >= 0, "Positive", "Negative"),
    max_abs_loading = max(abs(c(PC1_loading, PC2_loading, PC3_loading, PC4_loading)))
  ) %>%
  ungroup() %>%
  arrange(cluster_4, dominant_PC, desc(max_abs_loading))
```



```
write.csv(chemical_annotation, "chemical_data_reduced_pca_cluster_k4.csv", row.names = FALSE)
write.csv(scores_df, "PCA_Scores_by_Cheese.csv", row.names = F)
write.csv(loadings_df, "PCA_Loadings.csv", row.names = F)
```

Now we will move to the screening step.